Claude skills: choose, author and evaluate engineering workflows

Pick a workflow before collecting skills

A skill is reusable instruction for a task, not evidence that the task will be performed well. Four sources now connect community discovery, authoring practice, engineering evaluation and a concrete idea-to-ship workflow. The new Matt Pocock explainer expands a toolkit already named in the original collection, so its practical steps belong here rather than on a duplicate tile.

Start with a task, input, useful output and stopping condition. The official authoring guidance (https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices) is a reference for clear descriptions, concise operating instructions and supporting material. A brittle migration and a subjective design review need different levels of procedural detail.

Choose an inspected collection

The original directory included Superpowers (https://github.com/obra/superpowers), Matt Pocock skills (https://github.com/mattpocock/skills), UI/UX Pro Max (https://github.com/nextlevelbuilder/ui-ux-pro-max-skill) and Web Quality (https://github.com/addyosmani/web-quality-skills), plus specialized writing, diagrams and media resources retained in the individual source guide. Earlier replacement links were name-based matches after short links failed, not recovered redirects.

Inspect the actual repository and its install method. Do not assume every provider uses the same marketplace or that installing a whole collection improves results. For an existing team, start with one skill and one realistic acceptance task.

A concrete engineering path: decisions to evidence

The current Pocock repository (https://github.com/mattpocock/skills) offers a Claude plugin or editable file copies; use one route to avoid duplicates. Its currently documented Claude install is:

```
claude plugin install mattpocock-skills@claude-plugins-official
```

Run the setup skill once in a scratch repository. Choose local files when the exercise should not write to an external tracker.

```
/setup-matt-pocock-skills
/grill-with-docs
/to-spec
/to-tickets
/implement
/code-review
/retro
```

These commands identify a particular collection, not universal built-ins. They were checked against its current files, not installed or run here.

1. Resolve product decisions and domain terms. Read facts from the repository; ask the human about policy. "Pause" and "skip" can have different billing consequences even when both hide a delivery button.
2. Turn settled context into testable stories and agree where behavior will be tested. The spec skill (https://github.com/mattpocock/skills/blob/main/skills/engineering/to-spec/SKILL.md) is the reference for this step.
3. Create small vertical slices with explicit dependencies. A useful slice can demonstrate behavior across required layers. The ticket skill (https://github.com/mattpocock/skills/blob/main/skills/engineering/to-tickets/SKILL.md) supports that decomposition.
4. Implement the approved scope, then review both correctness against the stories and maintainability. Keep test results separate from the agent's narrative that it is finished.
5. Use a retrospective (https://github.com/mattpocock/skills/blob/main/skills/engineering/retro/SKILL.md) to suggest environment improvements. Select changes deliberately rather than allowing automatic instruction churn.

For a small edit, one short plan and one slice may be sufficient. The video's 150k-token rule is a heuristic, not a model-independent limit. Narrated completion time, stars and skill-token totals are not guarantees for another repository.

Author and evaluate your own skill

Define positive and negative trigger examples, missing-input cases and one realistic task with observable acceptance criteria. Check reference paths and script prerequisites. Compare the original and revised instructions on the same fixtures, recording false activations, omitted requirements and unsupported claims.

Remove instructions only when evidence supports the change. The older engineering reel's claim about an internal prompt reduction does not justify deleting a fixed percentage from every skill. Undocumented harness switches are not a stable public interface. Preserve important constraints even when a small test set fails to exercise them.

Practice project: one feature and its evidence pack

Use a fictional subscription app. Decide what happens to a packed delivery, when billing stops and when service resumes. Write a few acceptance cases at the billing boundary, then implement a small approved slice. Keep a spec, dependency map, test results, review notes and proposed retrospective changes together.

An alternative first milestone is a local skill audit: a manifest, trigger examples, reference checks and a before/after report. Choose this when the problem is instruction quality rather than application behavior. Neither project has been built as part of this guide.

Reusable brief

```
Evaluate one engineering skill on a bounded subscription-pause exercise.
Read facts from the code and ask about unresolved product decisions.
Settle packed-delivery behavior, billing timing and shared domain terms.
Write testable stories and minimal vertical tickets with dependency edges.
Compare baseline and skill-assisted runs on identical acceptance criteria.
Review behavior, reference paths, false activations and omitted requirements.
Suggest retrospective improvements for me to select, with rollback notes.
Do not publish issues, send a PR or connect real payments during the exercise.
```

What remains in the source material

Every original URL, transcript, directory entry and individual research PDF remains available. Dates and supported install paths can differ across those records. Use this unified task sequence as the entry point and the source-specific guides for historical detail.

Original sources

- https://www.instagram.com/p/DdcDA1sN8eS/
- https://www.instagram.com/reel/DeQYD7-CXLk/
- https://www.instagram.com/reel/DeCZn6JMmYf/
- https://youtu.be/Q3uuE2adUT8